Database Relationship Analysis using Joins

INNER JOIN | LEFT JOIN | RIGHT JOIN | Multi-table Reports

1. Objective

To combine the Customer, Product, Orders and Payment tables using SQL joins, retrieve complete order details, display customer purchase history and generate multi-table reports. An Order_Items bridge table links Orders and Product, because one order can contain many products and one product can appear in many orders.

2. Table Relationships

Customer (1) → (many) Orders | Orders (1) → (many) Order_Items | Product (1) → (many) Order_Items | Orders (1) → (many) Payment

Table	Primary Key	Foreign Key(s)	Purpose
Customer	customer_id	-	Customer details
Product	product_id	-	Product catalogue and price
Orders	order_id	customer_id	Order header (who, when)
Order_Items	(order_id, product_id)	order_id, product_id	Products and quantity in each order
Payment	payment_id	order_id	Payment made for an order

3. Join Types in Brief

Join	Returns	Use when
INNER JOIN	Only rows matching in both tables	You need confirmed relationships only
LEFT JOIN	All left rows + matches (NULL if none)	Keep every record from the main table
RIGHT JOIN	All right rows + matches (NULL if none)	Keep every record from the second table

4. Schema (DDL)

```
CREATE TABLE Customer (
    customer_id   INT PRIMARY KEY,
    name          VARCHAR(50) NOT NULL,
    email         VARCHAR(80),
    city          VARCHAR(50)
);

CREATE TABLE Product (
    product_id    INT PRIMARY KEY,
    product_name  VARCHAR(60) NOT NULL,
    category      VARCHAR(40),
    price         DECIMAL(10,2) NOT NULL
);

CREATE TABLE Orders (
    order_id      INT PRIMARY KEY,
    customer_id   INT,
    order_date    DATE NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
);

-- Bridge table: one order can have many products
CREATE TABLE Order_Items (
    order_id      INT,
    product_id    INT,
    quantity      INT NOT NULL,
    PRIMARY KEY (order_id, product_id),
    FOREIGN KEY (order_id)   REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Product(product_id)
);

CREATE TABLE Payment (
    payment_id    INT PRIMARY KEY,
    order_id      INT,
    amount        DECIMAL(10,2) NOT NULL,
    payment_mode  VARCHAR(20),
    payment_date  DATE,
    status        VARCHAR(15),
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)
);
```

5. Sample Data

Customer

customer_id	name	email	city
1	Arun	arun@mail.com	Chennai
2	Priya	priya@mail.com	Madurai
3	Karthik	karthik@mail.com	Coimbatore
4	Divya	divya@mail.com	Trichy
5	Suresh	suresh@mail.com	Chennai

Product

product_id	product_name	category	price
101	Laptop	Electronics	55000
102	Smartphone	Electronics	22000
103	Headphones	Accessories	2500
104	Office Chair	Furniture	7500
105	Smart Watch	Electronics	9000

Orders

order_id	customer_id	order_date
1001	1	2026-09-01
1002	2	2026-09-05
1003	1	2026-09-12
1004	3	2026-09-18
1005	5	2026-10-02
1006	NULL	2026-10-10

Order_Items

order_id	product_id	quantity
1001	101	1
1001	103	2
1002	102	1
1003	104	2
1004	103	1
1004	102	1
1005	101	1
1006	103	3

Payment

payment_id	order_id	amount	payment_mode	payment_date	status
1	1001	60000	Credit Card	2026-09-01	Paid
2	1002	22000	UPI	2026-09-05	Paid
3	1003	15000	Net Banking	2026-09-13	Paid
4	1004	24500	UPI	2026-09-18	Paid

Section 3. INNER JOIN

3a. Customers who placed orders

Returns only customers that have a matching order. Divya (no orders) and the orphan order 1006 are excluded.

```
SELECT c.customer_id, c.name, o.order_id, o.order_date
FROM Customer c
INNER JOIN Orders o ON c.customer_id = o.customer_id;
```

Output (5 rows):

customer_id	name	order_id	order_date
1	Arun	1001	2026-09-01
2	Priya	1002	2026-09-05
1	Arun	1003	2026-09-12
3	Karthik	1004	2026-09-18
5	Suresh	1005	2026-10-02

3b. Orders that have been paid

Only orders with a payment record appear. Orders 1005 and 1006 are unpaid, so they are dropped.

```
SELECT o.order_id, p.payment_id, p.amount, p.payment_mode
FROM Orders o
INNER JOIN Payment p ON o.order_id = p.order_id;
```

Output (4 rows):

order_id	payment_id	amount	payment_mode
1001	1	60000	Credit Card
1002	2	22000	UPI
1003	3	15000	Net Banking
1004	4	24500	UPI

Section 4. LEFT JOIN

4a. All customers with their orders

Every customer is kept. Divya has no order, so order_id is NULL.

```
SELECT c.customer_id, c.name, o.order_id
FROM Customer c
LEFT JOIN Orders o ON c.customer_id = o.customer_id;
```

Output (6 rows):

customer_id	name	order_id
1	Arun	1001
1	Arun	1003
2	Priya	1002
3	Karthik	1004
4	Divya	NULL
5	Suresh	1005

4b. Customers who have never ordered

LEFT JOIN plus WHERE ... IS NULL finds rows on the left with no match on the right.

```
SELECT c.customer_id, c.name
FROM Customer c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Output (1 row):

customer_id	name
4	Divya

4c. Orders with payment status

All orders are kept. COALESCE replaces NULL with 'Unpaid' where no payment exists.

```
SELECT o.order_id, p.amount, COALESCE(p.status, 'Unpaid') AS payment_status
FROM Orders o
LEFT JOIN Payment p ON o.order_id = p.order_id;
```

Output (6 rows):

order_id	amount	payment_status
1001	60000	Paid
1002	22000	Paid
1003	15000	Paid
1004	24500	Paid
1005	NULL	Unpaid
1006	NULL	Unpaid

Section 5. RIGHT JOIN

5a. All orders, including those without a customer

Every order is kept. Order 1006 has no customer, so name is NULL.

```
SELECT c.name, o.order_id, o.order_date
FROM Customer c
RIGHT JOIN Orders o ON c.customer_id = o.customer_id;
```

Output (6 rows):

name	order_id	order_date
Arun	1001	2026-09-01
Arun	1003	2026-09-12
Priya	1002	2026-09-05
Karthik	1004	2026-09-18
Suresh	1005	2026-10-02
NULL	1006	2026-10-10

5b. All products, including never-ordered ones

Every product is kept. Smart Watch has never been ordered, so order_id and quantity are NULL.

```
SELECT p.product_id, p.product_name, oi.order_id, oi.quantity
FROM Order_Items oi
RIGHT JOIN Product p ON oi.product_id = p.product_id;
```

Output (9 rows):

product_id	product_name	order_id	quantity
101	Laptop	1001	1
103	Headphones	1001	2
102	Smartphone	1002	1
104	Office Chair	1003	2
103	Headphones	1004	1
102	Smartphone	1004	1
101	Laptop	1005	1
103	Headphones	1006	3
105	Smart Watch	NULL	NULL

5c. Products that were never sold

RIGHT JOIN plus WHERE ... IS NULL finds products with no matching order item.

```
SELECT p.product_id, p.product_name
FROM Order_Items oi
RIGHT JOIN Product p ON oi.product_id = p.product_id
WHERE oi.order_id IS NULL;
```

Output (1 row):

product_id	product_name
105	Smart Watch

Section 6. COMPLETE ORDER DETAILS

6. Customer + Order + Product + Payment

Joins all five tables. LEFT JOIN keeps orders even if the customer or payment is missing; INNER JOIN on items and products ensures each row is a real purchased item.

```
SELECT o.order_id, o.order_date, c.name AS customer_name, c.city,
       pr.product_name, oi.quantity, pr.price,
       (oi.quantity * pr.price) AS line_total,
       COALESCE(pay.payment_mode, 'N/A') AS payment_mode,
       COALESCE(pay.status, 'Unpaid')    AS payment_status
FROM Orders o
LEFT JOIN Customer    c   ON o.customer_id = c.customer_id
INNER JOIN Order_Items oi ON o.order_id    = oi.order_id
INNER JOIN Product    pr  ON oi.product_id = pr.product_id
LEFT JOIN Payment     pay ON o.order_id    = pay.order_id
ORDER BY o.order_id, pr.product_name;
```

Output (8 rows):

order_id	order_date	customer_name	city	product_name	quantity	price	line_total	payment_mode	payment_status
1001	2026-09-01	Arun	Chennai	Headphones	2	2500	5000	Credit Card	Paid
1001	2026-09-01	Arun	Chennai	Laptop	1	55000	55000	Credit Card	Paid
1002	2026-09-05	Priya	Madurai	Smartphone	1	22000	22000	UPI	Paid
1003	2026-09-12	Arun	Chennai	Office Chair	2	7500	15000	Net Banking	Paid
1004	2026-09-18	Karthik	Coimbatore	Headphones	1	2500	2500	UPI	Paid
1004	2026-09-18	Karthik	Coimbatore	Smartphone	1	22000	22000	UPI	Paid
1005	2026-10-02	Suresh	Chennai	Laptop	1	55000	55000	N/A	Unpaid
1006	2026-10-10	NULL	NULL	Headphones	3	2500	7500	N/A	Unpaid

Section 7. CUSTOMER PURCHASE HISTORY

7a. Purchase history of one customer (Arun, id = 1)

Follows the chain Customer > Orders > Order_Items > Product for a single customer.

```
SELECT c.name, o.order_id, o.order_date, pr.product_name, oi.quantity,
       (oi.quantity * pr.price) AS item_total
FROM Customer c
INNER JOIN Orders o       ON c.customer_id = o.customer_id
INNER JOIN Order_Items oi ON o.order_id    = oi.order_id
INNER JOIN Product pr     ON oi.product_id = pr.product_id
WHERE c.customer_id = 1
ORDER BY o.order_date;
```

Output (3 rows):

name	order_id	order_date	product_name	quantity	item_total
Arun	1001	2026-09-01	Laptop	1	55000
Arun	1001	2026-09-01	Headphones	2	5000
Arun	1003	2026-09-12	Office Chair	2	15000

7b. Purchase history of all customers

LEFT JOINs throughout, so customers with no purchases (Divya) still appear.

```
SELECT c.customer_id, c.name, o.order_id, o.order_date, pr.product_name, oi.quantity
FROM Customer c
LEFT JOIN Orders o       ON c.customer_id = o.customer_id
LEFT JOIN Order_Items oi ON o.order_id    = oi.order_id
LEFT JOIN Product pr     ON oi.product_id = pr.product_id
ORDER BY c.customer_id, o.order_date;
```

Output (8 rows):

customer_id	name	order_id	order_date	product_name	quantity
1	Arun	1001	2026-09-01	Laptop	1
1	Arun	1001	2026-09-01	Headphones	2
1	Arun	1003	2026-09-12	Office Chair	2
2	Priya	1002	2026-09-05	Smartphone	1
3	Karthik	1004	2026-09-18	Smartphone	1
3	Karthik	1004	2026-09-18	Headphones	1
4	Divya	NULL	NULL	NULL	NULL
5	Suresh	1005	2026-10-02	Laptop	1

Section 8. MULTI-TABLE REPORTS

8a. Total spending per customer

Aggregates order count and total value per customer using joins with GROUP BY.

```
SELECT c.customer_id, c.name,
       COUNT(DISTINCT o.order_id)              AS total_orders,
       COALESCE(SUM(oi.quantity * pr.price),0) AS total_spent
FROM Customer c
LEFT JOIN Orders o       ON c.customer_id = o.customer_id
LEFT JOIN Order_Items oi ON o.order_id    = oi.order_id
LEFT JOIN Product pr     ON oi.product_id = pr.product_id
GROUP BY c.customer_id, c.name
ORDER BY total_spent DESC;
```

Output (5 rows):

customer_id	name	total_orders	total_spent
1	Arun	2	75000
5	Suresh	1	55000
3	Karthik	1	24500
2	Priya	1	22000
4	Divya	0	0

8b. Product sales report

Units sold and revenue per product, including products with zero sales.

```
SELECT pr.product_id, pr.product_name,
       COALESCE(SUM(oi.quantity), 0)            AS units_sold,
       COALESCE(SUM(oi.quantity * pr.price), 0) AS revenue
FROM Product pr
LEFT JOIN Order_Items oi ON pr.product_id = oi.product_id
GROUP BY pr.product_id, pr.product_name
ORDER BY revenue DESC;
```

Output (5 rows):

product_id	product_name	units_sold	revenue
101	Laptop	2	110000
102	Smartphone	2	44000
103	Headphones	6	15000
104	Office Chair	2	15000
105	Smart Watch	0	0

8c. Payment summary by mode

Number and total of payments for each payment mode.

```
SELECT pay.payment_mode, COUNT(*) AS no_of_payments, SUM(pay.amount) AS total_amount
FROM Payment pay
INNER JOIN Orders o   ON pay.order_id = o.order_id
INNER JOIN Customer c ON o.customer_id = c.customer_id
GROUP BY pay.payment_mode;
```

Output (3 rows):

payment_mode	no_of_payments	total_amount
Credit Card	1	60000
Net Banking	1	15000
UPI	2	46500

8d. Order value vs amount paid

Compares each order's total against its payment to find pending balances.

```
SELECT o.order_id, c.name,
       SUM(oi.quantity * pr.price)  AS order_total,
       COALESCE(MAX(pay.amount), 0) AS amount_paid,
       SUM(oi.quantity * pr.price) - COALESCE(MAX(pay.amount), 0) AS balance_due
FROM Orders o
LEFT JOIN Customer c      ON o.customer_id = c.customer_id
INNER JOIN Order_Items oi ON o.order_id    = oi.order_id
INNER JOIN Product pr     ON oi.product_id = pr.product_id
LEFT JOIN Payment pay     ON o.order_id    = pay.order_id
GROUP BY o.order_id, c.name
ORDER BY o.order_id;
```

Output (6 rows):

order_id	name	order_total	amount_paid	balance_due
1001	Arun	60000	60000	0
1002	Priya	22000	22000	0
1003	Arun	15000	15000	0
1004	Karthik	24500	24500	0
1005	Suresh	55000	0	55000
1006	NULL	7500	0	7500

8e. City-wise sales report

Orders and revenue grouped by customer city.

```
SELECT c.city, COUNT(DISTINCT o.order_id) AS orders,
       SUM(oi.quantity * pr.price) AS city_revenue
FROM Customer c
INNER JOIN Orders o       ON c.customer_id = o.customer_id
INNER JOIN Order_Items oi ON o.order_id    = oi.order_id
INNER JOIN Product pr     ON oi.product_id = pr.product_id
GROUP BY c.city
ORDER BY city_revenue DESC;
```

Output (3 rows):

city	orders	city_revenue
Chennai	3	130000
Coimbatore	1	24500
Madurai	1	22000

Conclusion

INNER JOIN gave only matched records, LEFT JOIN exposed customers without orders and orders without payments, and RIGHT JOIN exposed products never sold and an order with no customer. Combining all tables produced complete order details, customer purchase history and summary reports for customers, products, payments and cities.